

# Résumé de l'article

« *Greedy Function Approximation: A Gradient Boosting Machine* » — Jerome H. Friedman, *The Annals of Statistics*, 2001

Ce document résume l'article «Greedy Function Approximation: A Gradient Boosting Machine» de Jerome H. Friedman (Annals of Statistics, 2001), l'article fondateur qui a introduit le **gradient boosting**, dont XGBoost est l'une des implémentations les plus connues.

L'article part d'un problème classique d'estimation de fonction : on cherche une fonction  $F(x)$  qui prédit une réponse  $y$  à partir de variables explicatives  $x$ , en minimisant l'espérance d'une fonction de perte  $L(y, F(x))$ . Plutôt que de traiter ce problème comme une optimisation de paramètres (comme on le ferait pour un modèle paramétrique classique), Friedman propose de le voir comme une optimisation directement dans l'espace des fonctions. L'idée centrale est d'appliquer la méthode de la descente de gradient (steepest descent), habituellement utilisée en optimisation numérique de paramètres, à cet espace fonctionnel infini-dimensionnel.

Concrètement, on construit une approximation  $F(x)$  sous forme d'une somme additive de termes successifs (des « boosts »), chacun étant un petit ajustement qui pointe dans la direction du gradient négatif de la perte. Comme on ne dispose que d'un échantillon fini de données, ce gradient ne peut être calculé qu'aux points d'observation et ne se généralise pas directement à de nouvelles valeurs de  $x$ . La solution proposée consiste à entraîner, à chaque itération, un modèle simple (un « weak learner » ou « base learner », typiquement un arbre de régression) pour approximer ce gradient négatif par les moindres carrés. Ce modèle ainsi ajusté remplace le gradient théorique et permet de mettre à jour l'approximation courante via une recherche en ligne (line search) qui détermine le pas optimal. C'est l'algorithme générique 1 (« Gradient Boost ») du papier, qui constitue le cœur de toute la méthode.

Friedman décline ensuite cet algorithme générique pour plusieurs fonctions de perte usuelles. Pour la perte quadratique (moindres carrés, LS Boost), le pseudo-résidu à chaque étape est simplement le résidu courant : l'algorithme revient alors à ajuster successivement les résidus, ce qui sert de « test de cohérence » puisque cela retombe sur des méthodes connues. Pour la perte en valeur absolue (LAD, moins sensible aux valeurs extrêmes), le pseudo-résidu est le signe du résidu, et la mise à jour dans chaque feuille de l'arbre se fait par la médiane des résidus, ce qui rend la méthode très robuste. Pour la perte de Huber (M-regression), qui combine erreur quadratique près de zéro et erreur absolue au-delà d'un seuil  $\delta$  (estimé comme quantile des résidus courants), on obtient un compromis entre robustesse et efficacité statistique. Enfin, pour la classification, Friedman utilise la log-vraisemblance binomiale négative (logistique) à la fois en version binaire ( $L_K$  TreeBoost,  $K=2$ ) et multiclasse ( $K$  classes), ce dernier cas nécessitant l'induction de  $K$  arbres à chaque itération, un par classe, avec une approximation de Newton-Raphson pour les coefficients des feuilles. Une technique additionnelle, le « influence trimming », permet de supprimer les observations à faible poids (peu informatives à l'itération courante) pour accélérer considérablement les calculs, parfois en éliminant 90 à 95 % des données sans perte de précision.

Une section importante porte sur la régularisation. Le nombre d'itérations  $M$  est un premier paramètre de régularisation naturel : trop d'itérations entraîne du surapprentissage (overfitting). Mais Friedman montre empiriquement qu'une stratégie bien plus efficace consiste à introduire un taux d'apprentissage (learning rate)  $\nu$ , qui réduit chaque mise à jour d'un facteur  $0 < \nu \leq 1$  avant de l'ajouter au modèle (le fameux « shrinkage »). Plus  $\nu$  est petit, meilleure est généralement la performance finale, au prix d'un plus grand nombre d'itérations nécessaires. Ce shrinkage incrémental (appliqué à chaque étape) est beaucoup plus efficace qu'un simple rétrécissement global appliqué au modèle final, un résultat que Friedman qualifie lui-même de partiellement mystérieux. Le compromis  $\nu/M$  doit être ajusté conjointement, typiquement via un échantillon de validation.

L'article consacre également une section à la taille des arbres utilisés comme base learners. Le nombre de feuilles terminales  $J$  d'un arbre contrôle l'ordre maximal d'interaction entre variables que le modèle peut capturer (un arbre à  $J$  feuilles capture des interactions d'ordre au plus  $J-1$ ). Comme la plupart des fonctions rencontrées en pratique sont dominées par des interactions de faible ordre, des arbres relativement petits ( $J$  entre 4 et 8 typiquement) suffisent généralement et donnent de meilleurs résultats que des arbres plus grands, qui ajoutent du bruit sans gain de précision.

Une partie significative du document est dédiée à des études de simulation comparant les différentes variantes de TreeBoost (LS, LAD, M) entre elles et avec d'autres méthodes comme MARS, LogitBoost et AdaBoost.MH, sur une centaine de fonctions cibles générées aléatoirement. Les résultats montrent que LS TreeBoost est optimal pour des erreurs gaussiennes, mais s'effondre en présence d'erreurs à queue lourde (distribution « slash »), où LAD et M TreeBoost dominent largement. Le M TreeBoost apparaît comme un compromis robuste, performant presque aussi bien que le meilleur dans les deux cas extrêmes. Pour la classification multiclasse,  $L_K$  TreeBoost se montre globalement supérieur à LogitBoost et AdaBoost.MH lorsque le taux d'apprentissage n'est pas ajusté, mais les trois méthodes deviennent comparables une fois le shrinkage correctement calibré pour chacune.

Une autre contribution majeure du papier concerne l'interprétabilité des modèles boostés, souvent considérés comme des « boîtes noires » par rapport à un arbre unique. Friedman propose une mesure d'importance relative des variables, fondée sur la réduction cumulée de l'erreur (improvement) apportée par chaque variable lors des splits, moyennée sur tous les arbres du modèle. Il introduit également les graphiques de dépendance partielle (partial dependence plots), qui montrent comment la prédiction varie en fonction d'une ou deux variables, en moyennant sur la distribution marginale des autres variables. Pour les arbres de régression, ces graphiques peuvent être calculés efficacement sans repasser sur l'ensemble des données, via une traversée pondérée de l'arbre. Ces outils sont illustrés sur des fonctions simulées puis sur deux jeux de données réelles : un jeu de données géochimiques sur des grenats (prédiction de la concentration en titane) et un jeu de données démographiques (prédiction du revenu), montrant des interactions clairement détectables dans le premier cas et une structure essentiellement additive dans le second.

Le document se termine par une discussion sur les avantages de TreeBoost comme outil de data mining « prêt à l'emploi » : invariance à toute transformation monotone des

variables d'entrée (donc pas besoin de transformer les variables), robustesse native aux valeurs aberrantes (particulièrement pour LAD et M TreeBoost), gestion élégante des valeurs manquantes (héritée des arbres CART), et bonne scalabilité en fonction du nombre d'observations, de variables et d'itérations. Friedman note aussi que le boosting peut être repris là où il s'était arrêté si de nouvelles données arrivent, ou appliqué par blocs successifs si les données ne tiennent pas en mémoire.

**En résumé**, ce papier fondateur établit le gradient boosting comme un cadre général permettant d'optimiser n'importe quelle fonction de perte différentiable via une suite d'ajustements successifs par moindres carrés sur les pseudo-résidus (le gradient négatif de la perte), généralement combinés à des arbres de régression de petite taille (TreeBoost). Il introduit les deux principaux hyperparamètres encore utilisés aujourd'hui dans XGBoost et les bibliothèques de gradient boosting modernes — le nombre d'itérations  $M$  et le taux d'apprentissage (learning rate)  $\nu$  — ainsi que les outils d'interprétation (feature importance, partial dependence) devenus des standards de l'analyse des modèles d'arbres boostés.